第二次周报

本周主要学习了 PE 文件的区段头、导入表和导出表，并设计了 C++代码去实现解析 PE 的区段头、导入表和导出表。对 PE 文件在内存中的偏移建立更清晰的认识。

具体来看，PE 文件的结构是一层一层套起来的：DOS 头的 e_lfanew 字段指向 NT 头，NT 头由签名和文件头（也就是标准 PE 头）组成，文件头后面的可选 PE 头里放着 ImageBase、SectionAlignment、FileAlignment，以及数据目录表的位置和数量；数据目录表后面紧接着区段表，每个区段头描述一段数据的名称、在内存中的位置、在文件中的位置，以及是否可读、可写、可执行。

这里需要注意的是文件对齐和内存对齐并不相同，同一份数据在文件里和在内存里的偏移有差别，所以静态解析时必须先把 RVA 换算成 FOA，这也是后面解析导出表和导入表的前提。数据目录一共 16 项，本周用到的是导出表（索引 0）和导入表（索引 1），后面还会接触资源表、重定位表，以及下一阶段准备学习的 TLS 表（索引 9）。

实验工具：Visual Studio，PE-bear

实验一：解析区段头

（1）如何获取区段头，通过 DOS 头找到 NT 头，NT 头找到标准 PE 头，标准 PE 头中获取可选 PE 头的大小，通过代码中定义的 FileBuff 开始相加，找到区段头的首地址。这里使用的是更为方便的方法，用微软自带的宏来快速定位。

（2）区段头各字段的含义。一个区段头占 40 字节，常用的字段

有 Name（8 字节的区段名称）、VirtualSize（区段在内存中的实际大小）、VirtualAddress（区段在内存中的起始 RVA）、SizeOfRawData（区段在文件中对齐后的大小）、PointerToRawData（区段在文件中的起始偏移）和 Characteristics（区段是否可读、可写、可执行等属性）。其中 VirtualAddress 和 SizeOfRawData 正是后面做 RVA 转 FOA 时要用到的两个量。读取区段表时用微软提供的 IMAGE_FIRST_SECTION 宏可以直接定位到第一个区段头，比手工叠加偏移更省事。

```
FileSize = GetFileSize(hFile, 0);
FileBuff = new char[FileSize] {0};
DWORD RealReadBytes = 0;
```

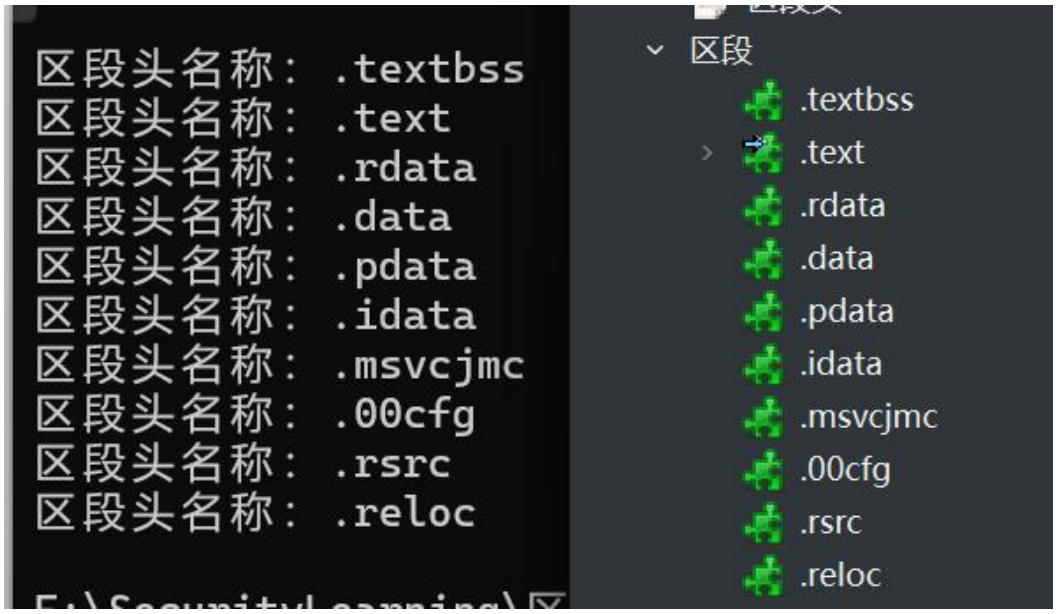
此处为 FileBuff 的定义

由于区段头的名称是 8 字节的 byte 型数组，因此不能直接 printf，这里把区段头名称放进一个数组内，用数组去表示。

```
void CPeUtil::PrintSectionHeaders()
{
    PIMAGE_SECTION_HEADER pSectionHeader = IMAGE_FIRST_SECTION(pNtHeader);
    for (int i = 0;i < pFileHeader->NumberOfSections;i++)
    {
        char name[9]{ 0 };
        memcpy_s(name, 9, pSectionHeader->Name, 8);
        printf("区段头名称：%s\n",name);
        pSectionHeader++;
    }
}
```

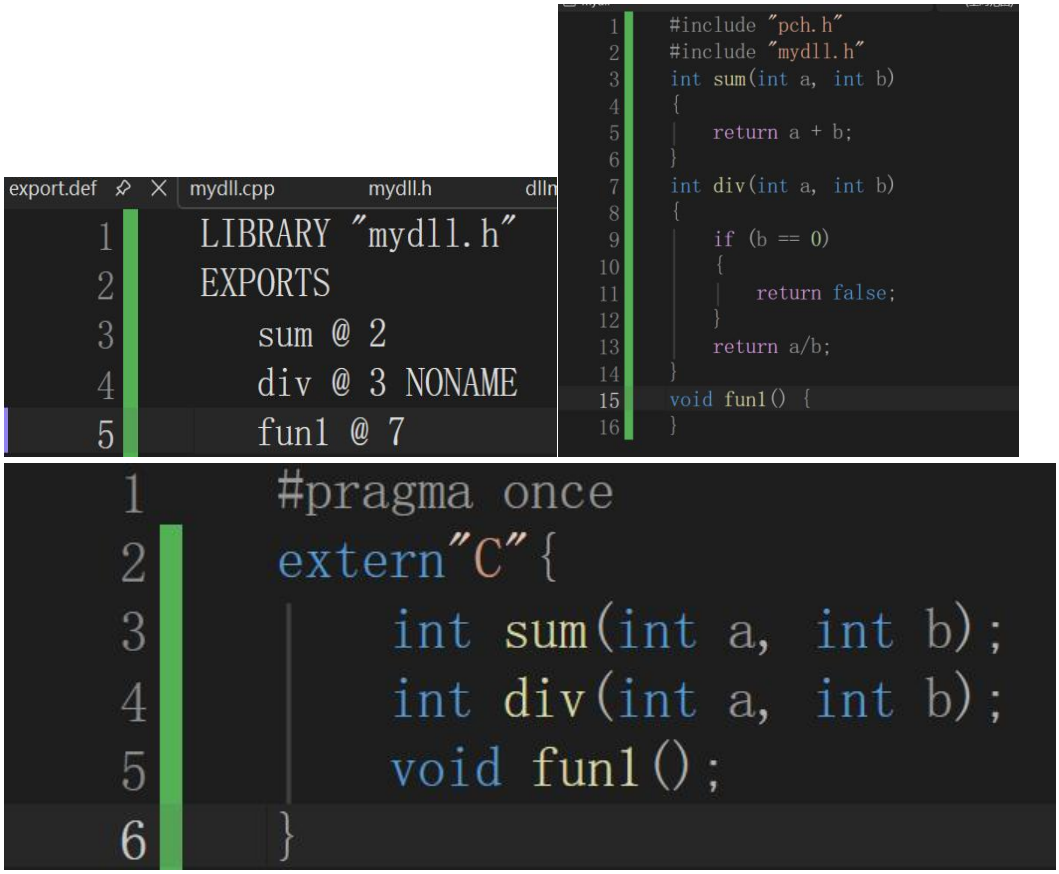
输出区段头名称

执行代码后，将提前准备好的 exe 文件放入 PE-bear 中解析，通过与工具解析出的和代码解析出的对比，发现实验成功



实验二：解析导出表

先新建一个 dll 文件，如图所示



共有 export.def\mydll.cpp\mydll.h

在这之前，要先建立一个将 RVA 转换成 FOA 的函数，由于 PE 文件在文件中和内存中对齐方式不同，导致加载方式不同。而 PE

文件解析给出的值都都是 RVA，因此想要静态解析需要将 RVA 转换成 FOA，由于导出表数据的 FOA 与区段的 FOA 的差值和数据的 RVA 与区段的 RVA 的差值一样，所以可用这个原理来转换。

导出表本身是 IMAGE_EXPORT_DIRECTORY 结构，最关键的是其中三个数组：AddressOfFunctions 是函数地址表，存的是各个函数的 RVA；AddressOfNames 是名称表，存的是函数名字符串的 RVA；AddressOfNameOrdinals 是序号表，存的是 WORD 类型的下标。名称表和序号表是一一对应的，同一个下标经序号表查到的值再去索引地址表，就能拿到对应函数的 RVA。结构里的 Base 是导出序号的起始值，NumberOfFunctions 和 NumberOfNames 分别记录地址表和名称表的项数，所以一个 DLL 既可以按名称导出，也可以按序号导出。



```
PIMAGE_SECTION_HEADER pSectionHeader = IMAGE_FIRST_SECTION(pNtHeader);
for (int i = 0;i < pFileHeader->NumberOfSections;i++)
{
    if ((rva >= pSectionHeader->VirtualAddress) &&
        (rva <= pSectionHeader->VirtualAddress + pSectionHeader->Misc.VirtualSize)
    {
        //数据的FOA=数据的RVA-区段的RVA+区段的FOA
        return rva-pSectionHeader->VirtualAddress+pSectionHeader->PointerToRawData
    }
    pSectionHeader++;
}
return 0;
```

源代码如下

```
{
    IMAGE_DATA_DIRECTORY directory = pOptionHeader->DataDirectory[0];
    PIMAGE_EXPORT_DIRECTORY pexort = (PIMAGE_EXPORT_DIRECTORY)
    (FileBuff + RvaToFoa(directory.VirtualAddress));
    char* dllName = RvaToFoa(pexort->Name) + FileBuff;
    printf("导出表名称：%s\n",dllName);
    DWORD* funaddr = (DWORD*)(RvaToFoa(pexort->AddressOfFunctions) + FileBuff);
    WORD* peot = (WORD*)(RvaToFoa(pexort->AddressOfNameOrdinals) + FileBuff);
    DWORD* pent = (DWORD*)(RvaToFoa(pexort->AddressOfNames) + FileBuff);
    for (int i = 0;i < pexort->NumberOfFunctions;i++)
    {
        printf("函数地址为：%x\n",*funaddr);
        for (int j = 0;j < pexort->NumberOfNames;j++)
        {
            if (peot[j] == i)
            {
                char* funName = (RvaToFoa(pent[j]) + FileBuff);
                printf("函数名称为：%s\n",funName);
                break;
            }
        }
        funaddr++;
```

通过运行，再把 dll 放到 PE-bear 中解析，经过比对，发现实验成功



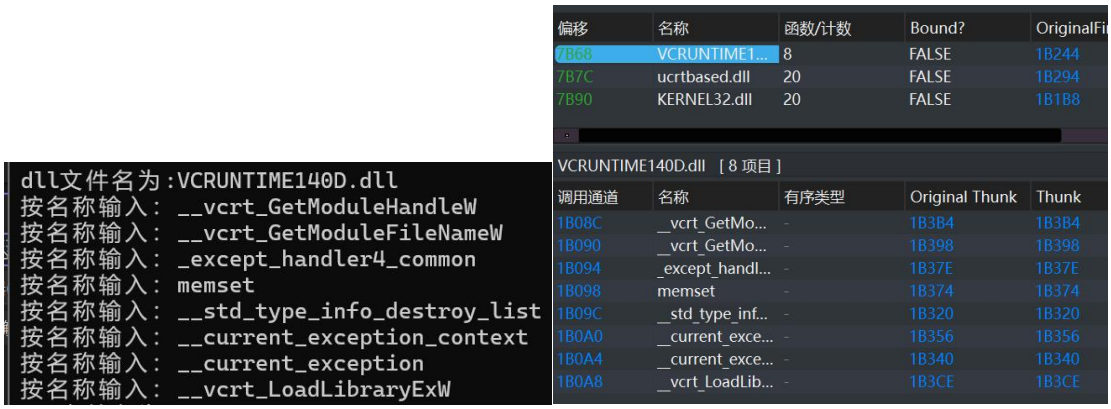
实验三：解析导入表

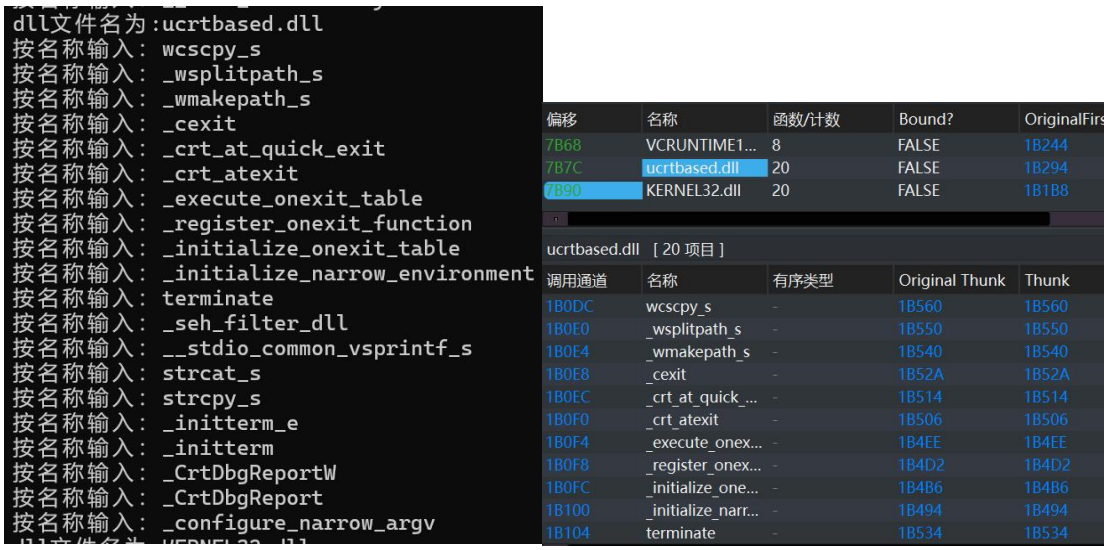
源码如下：

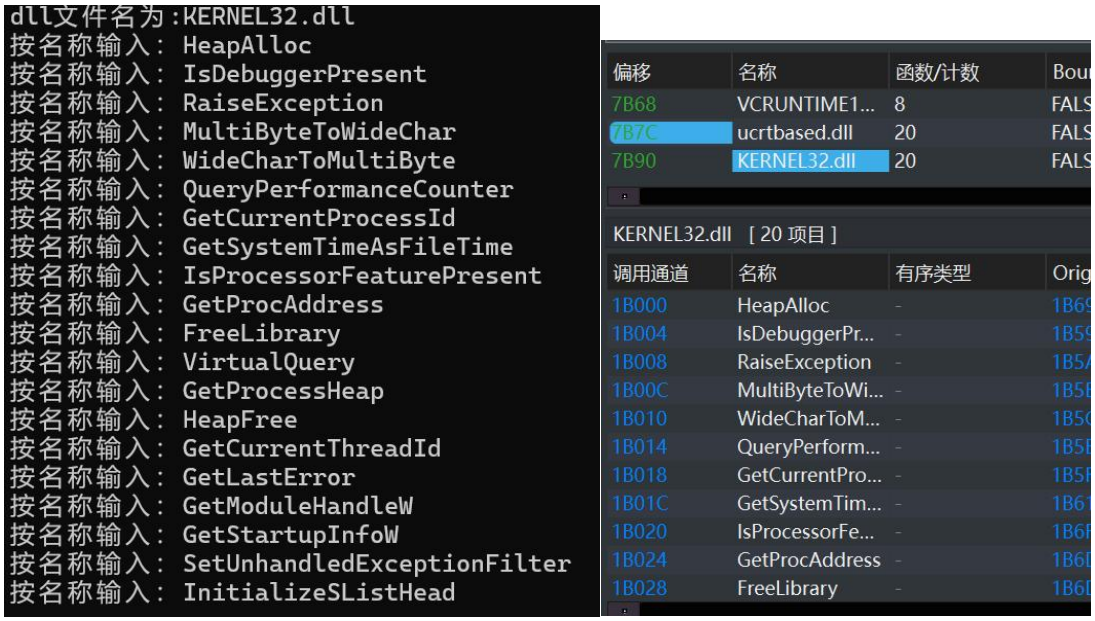
```
void CPeUtil::GetImportTable()
{
    IMAGE_DATA_DIRECTORY directory = pOptionHeader->DataDirectory[1];
    PIMAGE_IMPORT_DESCRIPTOR pImport = (PIMAGE_IMPORT_DESCRIPTOR)
        (RvaToFoa(directory.VirtualAddress) + FileBuff);
    while (pImport->OriginalFirstThunk)
    {
        char* dllName = RvaToFoa(pImport->Name) + FileBuff;
        printf("dll文件名为:%s\n",dllName);
        PIMAGE_THUNK_DATA pThunkData = (PIMAGE_THUNK_DATA)
            (RvaToFoa(pImport->OriginalFirstThunk) + FileBuff);
        while (pThunkData->u1.Function)
            //判断是否按序号导入
        {
            if (pThunkData->u1.Ordinal & 0x80000000)
            {
                printf("按序号输入: %d\n", pThunkData->u1.Ordinal & 0x7FFFFFFF);
            }
            else
            {
                PIMAGE_IMPORT_BY_NAME importName = (PIMAGE_IMPORT_BY_NAME)
                    (RvaToFoa(pThunkData->u1.AddressOfData) + FileBuff);
                printf("按名称输入: %s\n", importName->Name);
            }
            pThunkData++;
        }
            pImport++;
```

首先通过可选 PE 头找到数据表，获取导入表的起始地址，先导出 dll 文件名称，然后再判断导出类型，最高位是 0 或 1 来判断。最后运行得到导入表的解析。

导入表在数据目录里是一组 IMAGE_IMPORT_DESCRIPTOR，最后用一个全 0 的结构作为结束标志。每一项里的 OriginalFirstThunk 指向导入名称表（INT），FirstThunk 指向导入地址表（IAT），被导入的函数名就挂在 INT 指向的 IMAGE_IMPORT_BY_NAME 结构上，结构里包含 Hint 和 Name 两个字段。判断导入方式要看对应项的最高位：最高位是 1（IMAGE_ORDINAL_FLAG，值为 0x80000000）说明这个函数是按序号导入的，低 16 位就是序号；最高位是 0 则这一项是一个指向函数名字符串的 RVA，也就是按名称导入。







经过比对标准解析工具，发现导入表完全对应正确，实验成功。

四、下一阶段目标

结合本周的内容，下一阶段打算沿着从文件结构到内存行为这条

线继续，重点放在 TLS 反调试、堆和虚拟内存三个方向。

（1）TLS 反调试。先弄清楚 TLS 表的结构，它在数据目录里是索引 9，对应的结构是 IMAGE_TLS_DIRECTORY，重点是其中指向回调函数数组的 AddressOfCallBacks 字段。TLS 回调会在程序入口点之前被执行，这个时间点正是它常被用来做反调试和反沙箱的原因，所以要搞明白回调是怎么注册进去的、又是在什么时候被调用。初步目标是用 C++把 TLS 目录和回调地址解析出来，再用调试器验证回调的执行时机。

（2）堆。学习 Windows 堆的基本管理方式，包括堆的创建（HeapCreate）、内存的分配与释放（HeapAlloc、HeapFree，以及 C++里 new 和 malloc 背后的堆操作），理解堆区和堆块头的大致结构，并接触一下堆的调试手段，比如页堆（Page Heap）。初步目标是能用调试器观察一次内存申请背后的堆变化，为后面理解堆相关的漏洞利用打基础。

（3）虚拟内存。学习进程虚拟地址空间的布局、以页为单位的内存管理方式、页的保护属性（可读、可写、可执行），以及 VirtualAlloc、VirtualProtect 这类内存相关 API。这一块和本周的 PE 解析衔接得很紧，区段头里的 VirtualAddress 就是内存中的 RVA，PE 加载器正是按照 SectionAlignment 把各区段映射到虚拟内存里的。初步目标是能查看某个进程的内存区域分布，并解释 PE 文件加载之后在内存中的镜像结构。